```
{
public :
derived1()
{
cout << "\nConstructor of derived1 ...";
}
~derived1()
{
cout << "\nDestructor of derived2 ...";
}
};
int main()
{
derived1 x;
return 0;
}
```

வெளியீடு:

```
Constructor of base class...
Constructor of derived ...
Constructor of derived1 ...
Destructor of derived2 ...
Destructor of derived ...
Destructor of base class....
```

குறிப்பு:-

ஆக்கிகள் மரபுரிமம் பெற்ற இனக்குழுக்களின் வரிசையில் இயக்கப்படுகின்றன. அதாவது, அடிப்படை இனக்குழுவில் தொடங்கி இறுதியாக தருவிக்கப்பட்ட இனக்குழு வரையில் அழிப்பிகள் தலைகீழ்வரிசையில் இயக்கப்படும்.

தருவிக்கப்பட்ட இனக்குழுப் பொருளின் அளவு = அனைத்து அடிப்படை இனக்குழுவின் தரவு உறுப்புகளின் அளவு + தருவிக்கப்பட்ட இனக்குழுவில் உள்ள அனைத்துத் தரவு உறுப்புகளின் அளவு.

குறிப்பு:-

உறுப்பு செயற்கூறுகள், private என்று வகைப்படுத்தப்பட்ட உறுப்புகளை அணுகலாம்.

16.7 அடிப்படை இனக்குழுவின் செயற்கூறுகளை தருவிக்கப்பட்ட இனக்குழுவில் மேனிடல் / நிழலிடல் (Overriding / Shadowing)

மரபுரிமத்தில், சில சமயங்களில் அடிப்படை இனக்குழுக்களும், தருவிக்கப்பட்ட இனக்குழுக்களும் ஒரே பெயரில் உறுப்பு செயற்கூறினை பெற்றிருக்க முடியும். தருவிக்கப்பட்ட இனக்குழுவின் பொருள் பணிமிகுக்கப்பட்ட உறுப்பு செயற்கூறினை அழைக்கும் போது எந்த செயற்கூறினை செயல்படுத்துவது என்ற குழப்பத்தினை நிரல் பெயர்ப்பிக்கு ஏற்படுத்துகிறது. தருவிக்கப்பட்ட இனக்குழுவின் உறுப்பு செயற்கூறினுக்கு அடிப்படை இனக்குழு உறுப்பு செயற்கூறினை காட்டிலும் அதிக முன்னுரிமை உள்ளது. அடிப்படை இனக்குழுவின் உறுப்பு செயற்கூறுகள் தருவிக்கப்பட்ட இனக்குழுவின் உறுப்புச் செயற்கூறுகளின் பெயரை கொண்டிருந்தால், அது அடிப்படை இனக்குழுவின் உறுப்பு செயற்கூறுகளின் நிழலாக கருதப்படுகிறது. வரையெல்லை தீர்மானிப்பு செய்குறி (::) கொண்டு இந்த சிக்கலை தீர்க்கலாம்.

எடுத்துக்காட்டு நிரல் **16.11** தருவிக்கப்பட்ட இனக்குழுவில் வரையெல்லை தீர்மானிப்பு செயற்குறியின் பயன்பாட்டை விளக்குகிறது.

```
#include<iostream>
#include<string>
using namespace std;
class Employee
{
   private:
      char name[50];
      int code;
   public:
      void getdata();
      void display();
};
class staff: public Employee
{
   private:
      int ex;
   public:
         void getdata();
      void display();
};
void Employee::display()
{
         cout<<"Name:"<<name<<endl;
         cout<<"Code:"<<code<<endl;
}
void staff::display()
{
         Employee :: display();//overriding
         cout<<"Experience:"<<ex<<" Years"<<endl;
}
```


Chapter 16.indd 270 20-12-2022 15:34:42